

Guia completo — Criar a app do Clube Naval do Funchal (do início ao fim)

Guia de referência para construir a aplicação móvel (React Native + Expo) e o backend (Python / FastAPI), e ligá-los. Pensado para quem tem experiência de backend (JavaScript, Python, RPGLE) mas nunca criou uma app móvel. Ambiente assumido: **Windows + emulador Android no PC.**

Índice

- [Fase 0 — Mapa mental \(o que é o quê\)](#)
 - [Fase 1 — Instalar as ferramentas base](#)
 - [Fase 2 — Criar o projeto da app](#)
 - [Fase 3 — Meter o código que já existe](#)
 - [Fase 4 — O ajuste de navegação](#)
 - [Fase 5 — Criar o backend com Python / FastAPI](#)
 - [Fase 6 — Ligar a app ao backend](#)
 - [Fase 7 — O ciclo de trabalho diário](#)
 - [Fase 8 — Ordem sugerida para ligar os endpoints](#)
 - [Notas específicas \(vindo de RPGLE / bancos\)](#)
 - [Resumo do caminho completo](#)
 - [Próximos passos depois de tudo a funcionar](#)
-

Fase 0 — Mapa mental (o que é o quê)

Modelo mental para quem vem de backend:

- **React Native** — a framework para escrever a app (o código dos ecrãs que já está escrito).
- **Expo** — camada por cima do React Native que trata da parte difícil (compilação nativa, emulador, etc.). É o equivalente a "framework + build tool". O código existente é praticamente todo compatível.
- **Metro** — o bundler (como o webpack). Arranca um servidor de desenvolvimento; sempre que se grava um ficheiro, a app recarrega.
- **Emulador Android** — um telemóvel virtual no PC (vem do Android Studio).
- **Backend (Python / FastAPI)** — exatamente como qualquer API REST. A app faz `fetch()` aos endpoints. Nada de novo nesta parte.

A app e o backend são **dois projetos separados**, que correm em simultâneo durante o desenvolvimento.

Fase 1 — Instalar as ferramentas base (Windows)

1.1 — Node.js (LTS)

Descarregar o instalador LTS de <https://nodejs.org>. Confirmar depois no PowerShell:

```
powershell  
  
node --version  
npm --version
```

1.2 — Git

Para versionar o código: <https://git-scm.com>

1.3 — Editor

VS Code, se ainda não estiver instalado: <https://code.visualstudio.com>

1.4 — Android Studio (para o emulador)

- Descarregar de <https://developer.android.com/studio>
 - Na instalação, garantir que ficam marcados: **Android SDK**, **Android SDK Platform** e **Android Virtual Device**.
 - Abrir o Android Studio → **More Actions** → **Virtual Device Manager** → **Create Device** → escolher um telemóvel (ex: Pixel 7) → escolher uma imagem de sistema recente → **Finish**.
 - Isto cria o "telemóvel virtual". Pode ser arrancado a partir desse mesmo menu (botão play).
-

Fase 2 — Criar o projeto da app

No PowerShell, ir para a pasta de projetos e criar a app Expo:

```
powershell  
  
npx create-expo-app@latest clube-naval-app  
cd clube-naval-app
```

Quando perguntar o template, escolher **SDK 54 / default**.

Testar que arranca:

```
powershell
```

```
npx expo start
```

Com o emulador Android já aberto (Fase 1.4), carregar na tecla **(a)** no terminal — a app de exemplo abre no emulador. Se o ecrã de boas-vindas aparece, o ambiente está funcional. Parar com **(Ctrl+C)**.

Nota sobre versões: em junho de 2026 o SDK atual do Expo é o 54. Se aparecer um SDK mais recente no momento da instalação, seguir o template recomendado pelo próprio instalador.

Fase 3 — Meter o código que já existe

O template do Expo usa uma pasta **(app/)** com "file-based routing" (cada ficheiro = um ecrã). O código existente foi escrito no modelo clássico, com um **(App.js)** central que controla a navegação por estado. Para simplificar, usa-se o modelo clássico:

3.1 — Copiar o código

Copiar a pasta **(src/)** (todos os ecrãs e componentes entregues) para a raiz do projeto **(clubenaval-app/)**.

3.2 — Substituir o ponto de entrada

Apagar a pasta **(app/)** criada pelo template e criar um **(index.js)** na raiz com:

```
javascript
```

```
import { registerRootComponent } from 'expo';  
import App from './App';  
registerRootComponent(App);
```

Garantir que o **(App.js)** está na raiz. No **(package.json)**, confirmar que **(main)** aponta para **(index.js)**.

3.3 — Instalar dependências usadas pelo código

Com Expo, usar sempre **(npx expo install)** (escolhe a versão certa para o SDK):

```
powershell
```

```
npx expo install react-native-svg
```

(As restantes coisas usadas — **(View)**, **(Text)**, **(FlatList)**, **(Modal)**, **(Switch)**, **(StyleSheet)** — já vêm no React Native, não é preciso instalar nada.)

3.4 — Correr

`npx expo start` e carregar `(a)`. Deve aparecer o ecrã de login.

Fase 4 — O ajuste de navegação (importante)

No código entregue, a navegação é feita "à mão" com `useState` no `App.js` (`tela === 'home'`, etc.). Isto **funciona** e é ótimo para aprender, mas tem limitações: não tem botão "voltar" físico do Android nem histórico real.

Duas opções:

- **Opção A (começar simples):** deixar como está. Funciona para testar tudo. Avançar e tratar disto mais tarde.
- **Opção B (fazer bem):** migrar para **React Navigation**, a biblioteca padrão. Cada `setTela('x')` passa a ser `navigation.navigate('X')`. É um refactor mecânico e vale a pena, mas deve ser feito **depois** de já ter a app a correr e o backend ligado — uma coisa de cada vez.

Recomendação: Opção A agora, Opção B quando houver à-vontade.

Fase 5 — Criar o backend com Python / FastAPI

Projeto **separado** da app. Noutra pasta:

```
powershell

mkdir clube-naval-api
cd clube-naval-api
```

5.1 — Ambiente virtual

Isola as dependências (boa prática):

```
powershell

python -m venv venv
venv\Scripts\activate
```

Aparece `(venv)` no início da linha. Sempre que se volta ao projeto, correr `venv\Scripts\activate` de novo.

5.2 — Instalar FastAPI e o servidor

```
powershell
```

```
pip install fastapi "uvicorn[standard]"
```

- **FastAPI** — a framework (rotas, validação).
- **uvicorn** — o servidor que corre a aplicação (equivalente ao `node server.js`).

5.3 — Criar `main.py`

```
python
```

```

from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# Permite que a app (noutro "endereço") aceda à API.
# Em produção, trocar ["*"] pela origem real.
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# Exemplo: validar ID do clube (substitui o mock do LoginScreen)
@app.get("/membros/{clube_id}")
def get_membro(clube_id: str):
    # Aqui liga-se ao DB2/RPGLE ou à base de dados
    if clube_id == "CNF-2381":
        return {"nome": "Maria Sousa", "modalidade": "surf", "tipo": "aluno"}
    raise HTTPException(status_code=404, detail="não encontrado")

# Exemplo: próximas aulas de um atleta
@app.get("/aulas/proximas")
def aulas_proximas(idAtleta: str | None = None):
    return [
        {
            "id": "aula-1",
            "diaSemana": "quarta-feira",
            "dataCurta": "24/06",
            "hora": "17h00",
            "modalidade": "surf",
            "confirmada": False,
        }
    ]

```

5.4 — Arrancar o servidor

powershell

```
uvicorn main:app --reload --host 0.0.0.0 --port 3000
```

Dois pontos importantes nesse comando:

- `--reload` — recarrega sozinho quando se grava o ficheiro (como o `nodemon`).
- `--host 0.0.0.0` — **essencial** para o emulador conseguir chegar à API. Com o host por omissão (`127.0.0.1`), o emulador não a alcança.

5.5 — Testar

- No browser: `http://localhost:3000/membros/CNF-2381` → deve devolver o JSON.
- Bónus do FastAPI: abrir `http://localhost:3000/docs` → **documentação interativa automática** de todos os endpoints, onde se podem testar sem escrever código. Muito útil no dia-a-dia.

Esta parte é território conhecido — é uma API REST como as habituais, só que os clientes são telemóveis.

Fase 6 — Ligar a app ao backend (o ponto crítico)

Armadilha clássica a saber à partida: `localhost` no emulador não é o `localhost` do PC. O emulador é uma máquina própria. No emulador Android, usar o IP especial `10.0.2.2`, que o Android mapeia para o `localhost` da máquina anfitriã.

Tabela de referência rápida dos endereços, conforme onde a app corre:

Onde a app corre	Endereço a usar para a API
Emulador Android	<code>http://10.0.2.2:3000</code>
Telemóvel físico (mesma rede Wi-Fi)	<code>http://IP-DO-PC:3000</code> (ex: <code>192.168.1.50</code>)
Emulador iOS (em Mac)	<code>http://localhost:3000</code>

Criar um ficheiro de configuração único, `src/config.js`:

```
javascript
// Um só sítio para o endereço da API — fácil de trocar para produção depois.
export const API_URL = 'http://10.0.2.2:3000'; // emulador Android -> PC
```

Depois, nos sítios onde há funções `mock` (ex: `lookupClubeId` no `LoginScreen.js`), substituir pela chamada real:

```
javascript
```

```
import { API_URL } from '../config';

async function lookupClubeId(clubeId) {
  try {
    const res = await fetch(`${API_URL}/membros/${clubeId}`);
    if (!res.ok) return null;
    return await res.json(); // { nome, modalidade, tipo }
  } catch (e) {
    console.error('Erro de rede:', e);
    return null;
  }
}
```

Com o backend a correr (Fase 5) e a app a correr (Fase 3), o login passa a ir buscar dados reais ao FastAPI.

Fase 7 — O ciclo de trabalho diário

1. Abrir o emulador Android.
 2. **Terminal 1 (backend):** `cd clube-naval-api` → `venv\Scripts\activate` → `uvicorn main:app --reload --host 0.0.0.0 --port 3000`.
 3. **Terminal 2 (app):** `cd clube-naval-app` → `npx expo start` → tecla `(a)`.
 4. Editar, gravar, a app recarrega sozinha. O backend também (graças ao `--reload`).
 5. Para ver logs da app, abrir o menu de dev no emulador (`(Ctrl+M)`).
-

Fase 8 — Ordem sugerida para ligar os endpoints

Não fazer tudo de uma vez. Substituir os mocks por esta ordem (do mais simples ao mais complexo). Cada item é um endpoint FastAPI a criar do lado de Python e a respetiva troca do mock do lado da app:

1. `lookupClubeId` (login) — já feito na Fase 6.
2. **Autenticação real:** o `handleSubmit` do login chama `(POST /login)` com ID + password; o backend valida e devolve um **token** (JWT). A app guarda o token e envia-o nos pedidos seguintes no header `(Authorization)`.
3. Próximas aulas + eventos da `(HomeScreen)`.
4. Perfil (ler `(GET /perfil)` e gravar `(PUT /perfil)`).
5. Detalhe e confirmação de aulas.
6. Calendário (endpoint que devolve os tópicos por dia do mês).
7. Criação/edição de aulas e a lógica de recorrência.

8. Mensagens (o mais complexo — ver nota nas notas específicas).

Notas específicas (vindo de RPGLE / bancos)

Base de dados

O FastAPI não obriga a um ORM. Como já há experiência com DB2 e SQL, pode-se usar diretamente um driver (ex: `ibm_db` para DB2, ou `pyodbc`) e escrever o SQL à mão dentro das funções dos endpoints. Não é preciso aprender SQLAlchemy já — embora valha a pena mais tarde.

Validação de dados (Pydantic)

O FastAPI usa **Pydantic**: define-se a "forma" dos dados que se recebem/enviam com classes, e ele valida automaticamente. É parecido com declarar estruturas de dados fortemente tipadas, algo familiar do RPGLE:

```
python

from pydantic import BaseModel

class PerfilUpdate(BaseModel):
    nome: str
    telemovel: str
    email: str
    morada: str
    instagram: str | None = None # opcional, como definido no design

@app.put("/perfil")
def atualizar_perfil(dados: PerfilUpdate):
    # 'dados' já vem validado; se faltar um campo obrigatório,
    # o FastAPI devolve erro 422 automaticamente
    ...
```

Mensagens em tempo real

O chat (Fase 8.8) tem uma particularidade: para as mensagens aparecerem "ao vivo" sem o utilizador recarregar, o ideal é **WebSockets** (o FastAPI suporta nativamente, via `@app.websocket`). Mas começar simples: fazer primeiro com pedidos normais (a app vai buscar mensagens de X em X segundos — "polling") e só passar a WebSockets quando o resto estiver estável. Uma coisa de cada vez.

Modelo de recorrência das aulas (recordar a decisão de design)

As aulas recorrentes **não** são gravadas como N aulas físicas. Grava-se **um modelo de recorrência** (`(dias da semana)`, `(data de fim)`) que gera as ocorrências dinamicamente. Ao editar uma ocorrência, há 3 caminhos: alterar só aquela (exceção pontual), alterar todas (o

modelo inteiro), ou criar nova recorrência a partir daquela data (fecha o modelo atual no dia anterior e abre um novo). Isto tem impacto direto no desenho das tabelas.

Resumo do caminho completo

1. **Fases 0-1** — instalar Node, Android Studio, criar emulador.
 2. **Fase 2** — `npx create-expo-app@latest` e confirmar que arranca.
 3. **Fase 3** — copiar o `src/`, ligar o `App.js`, `npx expo install react-native-svg`.
 4. **Fase 4** — deixar a navegação por `useState` por agora; migrar para React Navigation mais tarde.
 5. **Fases 5-6** — montar o FastAPI, ligar a app via `10.0.2.2`.
 6. **Fases 7-8** — ciclo diário e substituição gradual dos mocks por endpoints reais.
-

Próximos passos depois de tudo a funcionar

- **Esquema da base de dados** — planear as tabelas (utilizadores, aulas, recorrências, confirmações de presença, eventos, participantes, mensagens, avisos). É a peça que dá mais retorno planear bem ao início.
 - **Autenticação com token (JWT)** — proteger os endpoints e identificar o utilizador (e o seu papel: aluno vs instrutor, que controla as versões diferentes dos ecrãs).
 - **Migrar a navegação** para React Navigation (Opção B da Fase 4).
 - **Notificações push** — para os avisos e mensagens chegarem ao telemóvel mesmo com a app fechada (Expo tem suporte próprio para isto).
 - **Build de produção** — quando estiver pronto para publicar, o Expo tem o serviço **EAS Build** para gerar o ficheiro instalável (Android `.aab` / iOS) para as lojas.
-

Documento de referência. Atualizar à medida que o projeto evolui.